

IoT - MQTT - Mosquitto - ESP32

MQTT y gestión de suscripciones en el broker

Guía técnica en español para comprender el modelo publish/subscribe, tópicos, filtros, QoS, sesiones, ACLs y una arquitectura práctica con ESP32, Raspberry Pi y Mosquitto.

Nivel: Ingeniería IoT / Redes de sensores

Enfoque: Mosquitto 2.x, ESP32 y Raspberry Pi 3B+

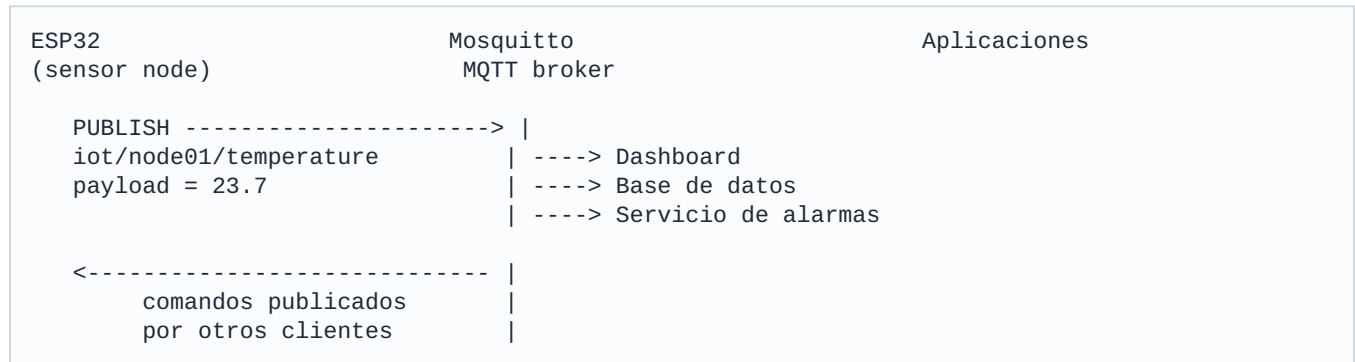
Contenido

1. Modelo de comunicación MQTT
2. Publisher, subscriber y broker
3. Tópicos y filtros de tópicos
4. Wildcards + y #
5. Cómo maneja el broker las suscripciones
6. Suscripción y desuscripción desde ESP32
7. Autenticación, autorización y suscripciones
8. ACLs en Mosquitto
9. QoS y su relación con las suscripciones
10. Sesiones persistentes y persistencia del broker
11. Mensajes retained
12. Inspección y administración en Mosquitto
13. Dynamic Security
14. Shared subscriptions
15. Arquitectura recomendada ESP32 + Raspberry Pi

Idea central: el cliente solicita una suscripción; el broker la almacena, verifica si está autorizada y enruta las publicaciones que coinciden con el filtro.

1. Modelo de comunicación MQTT

MQTT es un protocolo de mensajería orientado a **publish/subscribe**. A diferencia de un esquema HTTP típico, un dispositivo no necesita conocer directamente a todos los consumidores de sus datos. Publica información en un **tópico** y el broker distribuye el mensaje a los clientes que estén suscritos a filtros compatibles.



El broker desacopla productores y consumidores. El ESP32 no necesita saber si existen cero, uno o cien suscriptores.

Consecuencia importante: publisher y subscriber no son roles permanentes. Un mismo ESP32 puede publicar telemetría y, al mismo tiempo, suscribirse a comandos.

2. Publisher, subscriber y broker

Publisher

Publica mensajes en un t pico espec fico.

iot/node01/sensor/temperature
23.7

Subscriber

Solicita recibir mensajes usando un filtro de t pico.

iot/node01/sensor/#

Broker

Recibe publicaciones, mantiene el estado de las suscripciones, aplica autenticaci n/autorizaci n y entrega cada mensaje a los clientes cuyo filtro coincide.

Ejemplo de tabla conceptual del broker

Client ID	Filtro de suscripci�n	QoS solicitado
esp32-node01	iot/node01/command/#	1
dashboard	iot/+sensor/#	0
logger	iot/#	1

Esta tabla es conceptual: normalmente no se edita a mano en Mosquitto. Se construye como resultado de paquetes MQTT **SUBSCRIBE** recibidos desde los clientes.

3. Tópicos y filtros de tópicos

Un **tópico** identifica dónde se publica un mensaje. Un **topic filter** define qué conjunto de tópicos desea recibir un subscriber.

Concepto	Ejemplo	Uso
Tópico de publicación	iot/node01/sensor/temperature	Usado en PUBLISH
Filtro de suscripción	iot/node01/sensor/+	Usado en SUBSCRIBE
Filtro amplio	iot/node01/#	Recibe todos los niveles inferiores

Regla práctica: los wildcards + y # se usan en filtros de suscripción; no deben utilizarse como nombre normal de un tópico al publicar.

Jerarquía recomendada

```
iot/  
+-- node01/  
|   +-- sensor/  
|   |   +-- temperature  
|   |   +-- humidity  
|   |   +-- pressure  
|   +-- network/  
|   |   +-- rssi  
|   +-- status  
|   +-- command/  
|       +-- led  
|       +-- sampling  
|       +-- restart  
+-- node02/  
    +-- ...
```

4. Wildcards de suscripción

Wildcard **+**

Representa exactamente **un nivel** de la jerarquía.

```
iot/+/temperature
```

Coincide con:

```
iot/node01/temperature  
iot/node02/temperature  
iot/greenhouse1/temperature
```

No coincide con:

```
iot/node01/sensor/temperature
```

Wildcard **#**

Representa todos los niveles restantes a partir de ese punto.

```
iot/node01/#
```

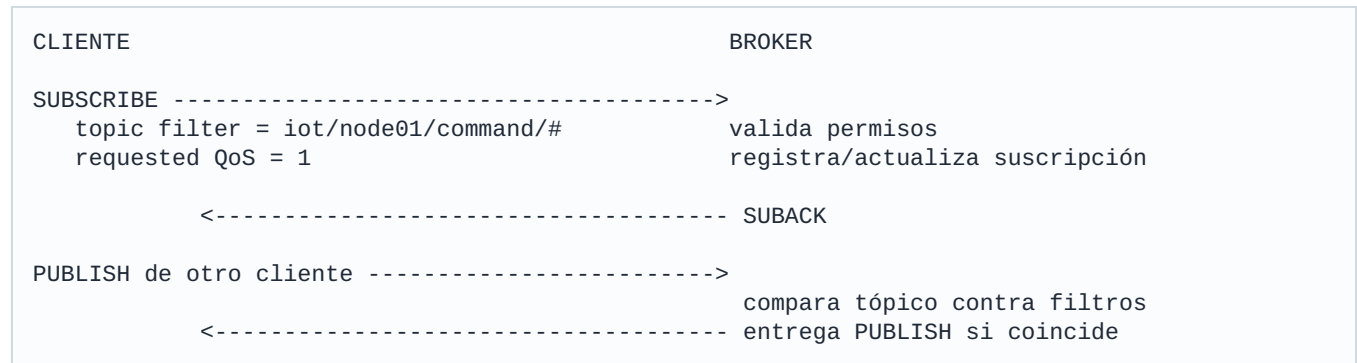
Puede recibir, por ejemplo:

```
iot/node01/temperature  
iot/node01/sensor/temperature  
iot/node01/network/rssi  
iot/node01/command/restart
```

Para depuración, una suscripción a `iot/#` permite observar todo el árbol de tópicos bajo `iot/`.

5. Cómo maneja el broker las suscripciones

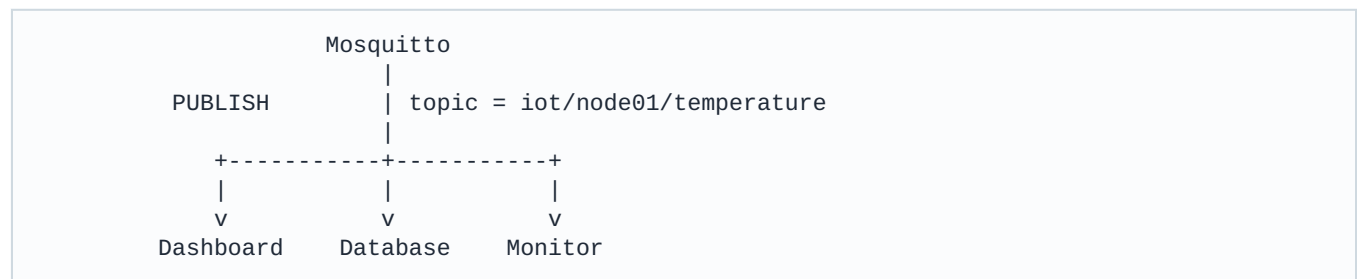
La secuencia básica es:



El broker mantiene la relación entre el cliente/sesión y sus filtros. Cuando llega una publicación, realiza **topic matching** contra los filtros activos.

Múltiples suscriptores

Si tres clientes tienen filtros compatibles, los tres reciben su propia copia lógica del mensaje.



6. Suscripción y desuscripción desde ESP32

Con la biblioteca Arduino **PubSubClient**:

```
mqttClient.subscribe("iot/node01/command/#");
```

El broker responde con un SUBACK. Para procesar mensajes recibidos se define un callback:

```
void callback(char* topic, byte* payload, unsigned int length)
{
    Serial.print("Topic: ");
    Serial.println(topic);

    Serial.print("Payload: ");
    for (unsigned int i = 0; i < length; i++) {
        Serial.print((char)payload[i]);
    }
    Serial.println();

    mqttClient.setCallback(callback);
}
```

Para eliminar una suscripción:

```
mqttClient.unsubscribe("iot/node01/command/#");
```

En un ESP32 típico, `mqttClient.loop()` debe ejecutarse periódicamente para procesar tráfico entrante, keep-alive y callbacks.

7. Autenticación, autorización y suscripciones

Son tres mecanismos distintos:

Mecanismo	Pregunta que responde	Ejemplo en Mosquitto
Autenticación	¿Quién eres?	password_file
Autorización	¿A qué tópicos puedes acceder?	acl_file
Suscripción	¿Qué mensajes quieres recibir ahora?	SUBSCRIBE del cliente

Una ACL no crea una suscripción. Solo indica si el cliente está autorizado a publicar o recibir en determinados tópicos.

Configuración básica

```
listener 1883
allow_anonymous false
password_file /etc/mosquitto/passwd
acl_file /etc/mosquitto/acl
```

En el escenario ESP32 + Raspberry Pi, el broker autentica cada nodo y las ACLs pueden impedir que un dispositivo publique datos dentro del espacio de otro dispositivo.

8. ACLs en Mosquitto

Ejemplo para un nodo específico:

```
user esp32_node01

topic write iot/node01/sensor/#
topic write iot/node01/status/#
topic read iot/node01/command/#
```

Interpretación:

- **write:** el nodo puede publicar telemetría y estado.
- **read:** puede recibir comandos dirigidos a su espacio.
- No queda autorizado automáticamente para acceder a `iot/node02/...`

Patrones escalables con %u y %c

Para muchos nodos puede ser más eficiente asociar la jerarquía de tópicos al usuario o al Client ID.

```
pattern write iot/%u/sensor/#
pattern read iot/%u/command/#
```

Si el usuario MQTT es `node01`, el patrón lo vincula a `iot/node01/...`

%u representa el username; **%c** representa el Client ID. La sintaxis exacta y las capacidades pueden variar según la versión/configuración de Mosquitto, por lo que conviene verificarlas en la documentación de la versión instalada.

9. QoS y su relación con la suscripción

MQTT define tres niveles de Quality of Service:

QoS	Semántica simplificada	Uso típico
0	Entrega como máximo una vez; sin retransmisión MQTT.	Telemetría frecuente tolerante a pérdidas.
1	Entrega al menos una vez; puede haber duplicados.	Comandos, eventos o alarmas importantes.
2	Entrega exactamente una vez a nivel MQTT, con mayor intercambio de control.	Casos donde el costo extra se justifica.

El subscriber solicita un QoS máximo al crear la suscripción. El QoS efectivo de la entrega queda limitado por el QoS usado por el publisher y por el máximo de la suscripción.

QoS PUBLISH	QoS suscripción	QoS de entrega
0	1	0
1	1	1
2	1	1

Recomendación inicial para IoT: sensores periódicos en QoS 0; comandos y eventos importantes en QoS 1. QoS 2 suele ser innecesario en nodos pequeños salvo requerimientos específicos.

10. Sesiones persistentes y persistencia del broker

Una suscripción pertenece a una **sesión MQTT**. Si la sesión es persistente, el broker puede conservar su estado cuando se pierde temporalmente la conexión de red.

```
ESP32 conectado
|
+-- SUBSCRIBE iot/node01/command/#
|
X Wi-Fi se pierde
|
+-- Broker conserva la sesión (si fue configurada como persistente)
|
ESP32 reconecta con el mismo Client ID
|
+-- Recupera el contexto de la sesión según las reglas MQTT
```

No confundir dos conceptos

Concepto	Qué conserva
Sesión MQTT persistente	Estado de la sesión del cliente, incluyendo suscripciones y mensajes pendientes aplicables.
Persistencia de Mosquitto en disco	Permite que ciertos estados del broker sobrevivan a un reinicio del proceso/host.

Sesión persistente MQTT y persistencia del broker en disco se relacionan, pero no son lo mismo.

11. Mensajes retained

Un mensaje publicado con la bandera **retain** hace que el broker conserve el último valor retained para ese tópico.

```
ESP32 -- PUBLISH retain=true --> Mosquitto
      topic: iot/node01/status
      payload: online
```

Tiempo después...

```
Dashboard -- SUBSCRIBE -----> Mosquitto
      iot/node01/status
```

Dashboard <----- recibe inmediatamente el valor retained

Esto es útil para estado actual, configuración deseada o última lectura representativa.

Mecanismo	Qué recuerda
Suscripción	Qué tópicos/filtros desea recibir un cliente.
Retained message	El último mensaje retained asociado a un tópico.
Sesión persistente	Estado asociado a la sesión de un cliente.

12. Inspección y administración en Mosquitto

Observar todos los tópicos de una rama

```
mosquitto_sub \  
-h localhost \  
-u esp32 \  
-P 'PASSWORD' \  
-t 'iot/#' \  
-v
```

Observar estadísticas del broker

```
mosquitto_sub \  
-h localhost \  
-u esp32 \  
-P 'PASSWORD' \  
-t '$SYS/#' \  
-v
```

Los tópicos `$SYS/...` pueden exponer métricas y estado del broker según su configuración.

MQTT no define un comando administrativo estándar del tipo "listar todas las suscripciones activas". La visibilidad exacta depende del broker, logging y herramientas de administración disponibles.

Logs

```
sudo journalctl -u mosquitto -f  
sudo tail -f /var/log/mosquitto/mosquitto.log
```

13. Dynamic Security

Para sistemas más grandes, editar manualmente `passwd` y `acl` puede volverse difícil de mantener.

Mosquitto dispone de un mecanismo/plugin de **Dynamic Security** que permite administrar clientes, grupos, roles y reglas de acceso de manera más dinámica.

```
Mosquitto
|
+-- Clientes
+-- Grupos
+-- Roles
+-- ACLs
    |
    +-- sensor-node
    +-- dashboard
    +-- backend-worker
```

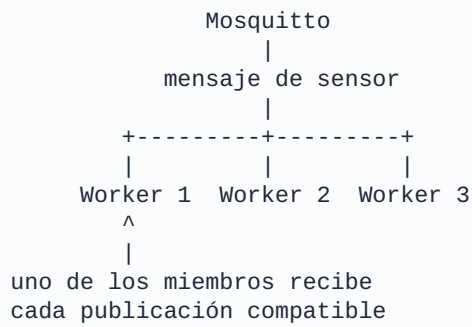
Esto es especialmente útil cuando aumenta el número de ESP32 y se desea separar la administración de identidades y permisos del archivo de configuración estático.

Para un laboratorio o una red pequeña, `password_file` + `acl_file` sigue siendo una solución clara y suficiente. Dynamic Security resulta más atractivo cuando la administración debe cambiar en ejecución o escalar a muchos clientes.

14. Shared subscriptions

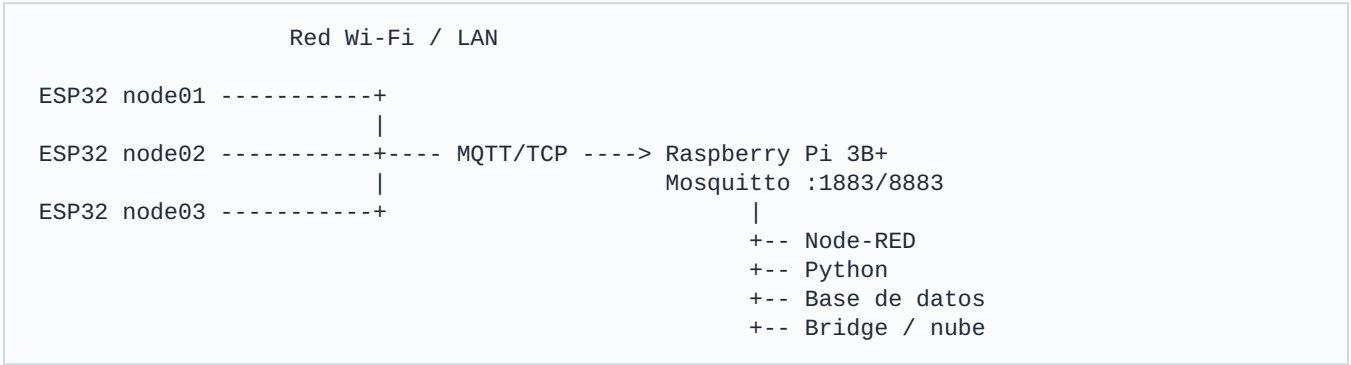
Una suscripción compartida permite que varios consumidores formen un grupo de trabajo. En lugar de entregar cada publicación a todos los consumidores del grupo, el broker distribuye cada mensaje compatible a uno de ellos.

```
$share/workers/iot/+/sensor/#
```



Es útil para balancear carga entre procesos backend. Normalmente no es la primera característica necesaria en un ESP32 simple, pero sí puede ser útil para pipelines de procesamiento.

15. Arquitectura recomendada para ESP32 + Raspberry Pi



Convención propuesta

Entidad	PUBLISH	SUBSCRIBE
ESP32 node01	iot/node01/sensor/# iot/node01/network/# iot/node01/status	iot/node01/command/#
Dashboard	iot/+/command/...	iot/+/sensor/# iot/+/network/# iot/+/status

ACL conceptual

```
user node01

topic write iot/node01/sensor/#
topic write iot/node01/network/#
topic write iot/node01/status
topic read iot/node01/command/#
```

Diseño recomendado: cada dispositivo tiene sus propias credenciales y solo puede publicar dentro de su namespace y recibir comandos dirigidos a él.

16. Resumen de conceptos

Elemento	Responsabilidad
Publisher	Publica datos en tópicos específicos.
Subscriber	Solicita mensajes mediante filtros de tópico.
Broker	Recibe publicaciones, mantiene suscripciones y realiza topic matching.
password_file	Autenticación de clientes.
ACL	Autorización de acceso a tópicos.
QoS	Controla garantías de entrega y overhead.
Retained	Conserva el último valor retained de un tópico.
Sesión persistente	Conserva estado de la sesión del cliente entre conexiones.
Shared subscription	Permite repartir mensajes entre consumidores de un grupo.

Concepto final: el cliente decide a qué filtros desea suscribirse; el broker decide si esa operación está autorizada, mantiene la suscripción dentro de la sesión y entrega las publicaciones que coinciden.

Documento preparado como material técnico de apoyo para experimentación IoT con ESP32, Raspberry Pi y Mosquitto.